NOTE PERSONNELLE · TECHNIQUE

Assistant IA de SmartTrans-AI

Fonctionnement métier et technique : ancrage sur PostgreSQL, mode GROUNDED, mode LLM optionnel.

1. Vue fonctionnelle

L'assistant est un chat d'exploitation, pas un planificateur automatique de tournées. L'admin ou le responsable pose une question en français. L'outil répond à partir des missions, chauffeurs, véhicules et clients déjà en base. Il peut aussi signaler des échéances (permis, assurance, visite technique) et suggérer une ressource disponible.

Qui a le droit ?

Uniquement ADMIN et TRANSPORT_MANAGER (`@PreAuthorize` sur `AssistantController`). Le chauffeur n'a pas le menu ni l'API : un appel donne 403.

Ce qu'on peut demander

Intention	Exemples	Comportement attendu
Effectif	« Combien de missions ? »	Réponse courte : un nombre + le mot (ex. « 3 missions »), sans roman.
Recherche	Nom, immat, référence	Extraits des fiches trouvées + citations cliquables vers l'écran métier.
Situation	résumé, dashboard, risque	Vue d'ensemble : totaux, permis / documents à 30 jours.
Suggestion	qui est dispo, affecter…	Chauffeur AVAILABLE (permis OK) + véhicule AVAILABLE (assurance / CT OK).

Règle métier importante : ne pas inventer un chauffeur ou une immatriculation absents du contexte. Si rien n'est trouvé, l'assistant le dit clairement.

2. Architecture technique

Front : `AssistantPage` → `POST /api/v1/assistant/ask { message }` (JWT obligatoire). Back : AssistantServiceImpl.ask en lecture seule (transaction readOnly).

```
Utilisateur (admin / responsable)
        │  question en français
        ▼
React  POST /assistant/ask  + Bearer JWT
        │
        ▼
AssistantServiceImpl
        │  1. sanitize (enlève % et _)
        │  2. collect()  → AssistantGrounding  (requêtes SQL)
        │  3. si clé OpenAI  → OpenAiChatClient.complete(system, contexte)
        │  4. sinon / échec  → GroundedAnswerComposer.extractive()
        ▼
Réponse { answer, mode: "LLM" | "GROUNDED", citations[] }
```

3. L'ancrage (grounding) — le cœur du dispositif

Sans ancrage, un modèle de langage hallucine des effectifs. Ici, collect() construit un objet `AssistantGrounding` à partir de PostgreSQL avant toute génération.

Découpage de la question

- needles : tokens ≥ 3 lettres, hors mots vides (le, la, quel…). Si la phrase est courte, la phrase entière aussi.
- Pour chaque needle : search missions, chauffeurs, véhicules (immat), clients (nom), max 8 par type.
- overview si la question parle de résumé, dashboard, alerte, risque, état…
- suggest si elle parle de disponible, recommander, affecter, quel chauffeur / véhicule…
- Si overview ou suggest : charger aussi permis / documents expirant sous 30 jours, ressources AVAILABLE, missions planifiées incomplètes.

4. Deux modes de réponse

Mode	Quand	Mécanisme
GROUNDED	Pas de clé API, ou appel LLM en échec / vide	Règles Java dans GroundedAnswerComposer : décompte, suggestion, synthèse, ou « je n'ai pas trouvé ».
LLM	app.assistant.openai-api-key renseignée et HTTP OK	Contexte textuel (contextBlock) + prompt système envoyés à /chat/completions. Température 0.2. Modèle par défaut gpt-4o-mini.

Prompt système (contrainte)

Le modèle doit répondre uniquement à partir du contexte : décompte = nombre + mot ; recommandation = chauffeur AVAILABLE permis valide + véhicule AVAILABLE documents valides, une phrase de justification ; français, concis ; rien inventer.

Même en mode LLM, les citations (MISSION, DRIVER, VEHICLE, CLIENT) sont calculées côté serveur à partir du grounding, pas par le modèle. Le front s'en sert pour ouvrir la fiche.

5. Composer extractif (sans LLM)

- Si la question est un décompte → `countOnlyAnswer` (missions totales / planifiées / en cours).
- Si suggest → bloc chauffeur + véhicule disponibles, sinon message d'indisponibilité.
- Si overview ou pas de hits mais des totaux / alertes → phrase d'état opérationnel + échéances 30 jours.
- Sinon liste courte des objets trouvés ; si vraiment rien : phrase d'échec honnête.

6. Configuration et limites

Élément	Détail
Clé / modèle / URL	AssistantProperties : openaiApiKey, openaiModel, openaiBaseUrl (défaut api.openai.com/v1).
Activation LLM	enabled() = clé non vide. Sinon GROUNDED, zéro appel réseau.
Robustesse	Exception HTTP → log warn, Optional.empty() → repli GROUNDED.

Élément	Détail
Périmètre	Pas d’optimisation de tournées, pas d’écriture en base via le chat.
Qualité	Liée au contexte injecté (plafond 8 fiches, 5 dispos). Une question trop vague = peu de needles.

Fichiers à relire

`AssistantServiceImpl`, `GroundedAnswerComposer`, `OpenAiChatClient`, `AssistantController`, `AssistantPage.tsx`.

En une phrase pour un jury : l’IA n’est pas branchée à vide ; elle ne parle que de ce que PostgreSQL vient de lui donner, et si le prestataire LLM est absent, des règles déterministes répondent quand même.